

第6章 用CUDA进行GPU编程

计算机与软件学院 陆克中

路线图

- GPU和GPGPU
- GPU架构
- CUDA hello
- 线程、线程块和线程网格向量加法
- 从CUDA核函数返回结果
- CUDA梯形法则I
- CUDA梯形法则II：提升性能
- 用warpSize个线程块实现梯形法则
- CUDA梯形法则 III：使用具有多个线程束的线程块
- 双调排序

GPU和GPGPU

■ 图形处理单元 (GPU)

- ▶ 视频游戏和视频动画。
- ▶ 提高渲染性能。

■ 图形处理单元通用计算 (GPGPU)

- ▶ 用GPU解决一般的计算问题，如搜索和排序问题。
- ▶ 早期只能使用计算机图形API进行编程，如Direct3D、OpenGL等。
 - 程序开发非常复杂。

GPU和GPGPU

- 在GPU上进行通用编程的API。
 - ▶ 更接近于传统高级CPU语言。
 - ▶ CUDA
 - 适用于NVIDIA的GPU，设置相对较少。
 - ▶ OpenCL
 - 高度可移植，用于任意的GPU和其他处理器FPGA、DSP等。

GPU架构

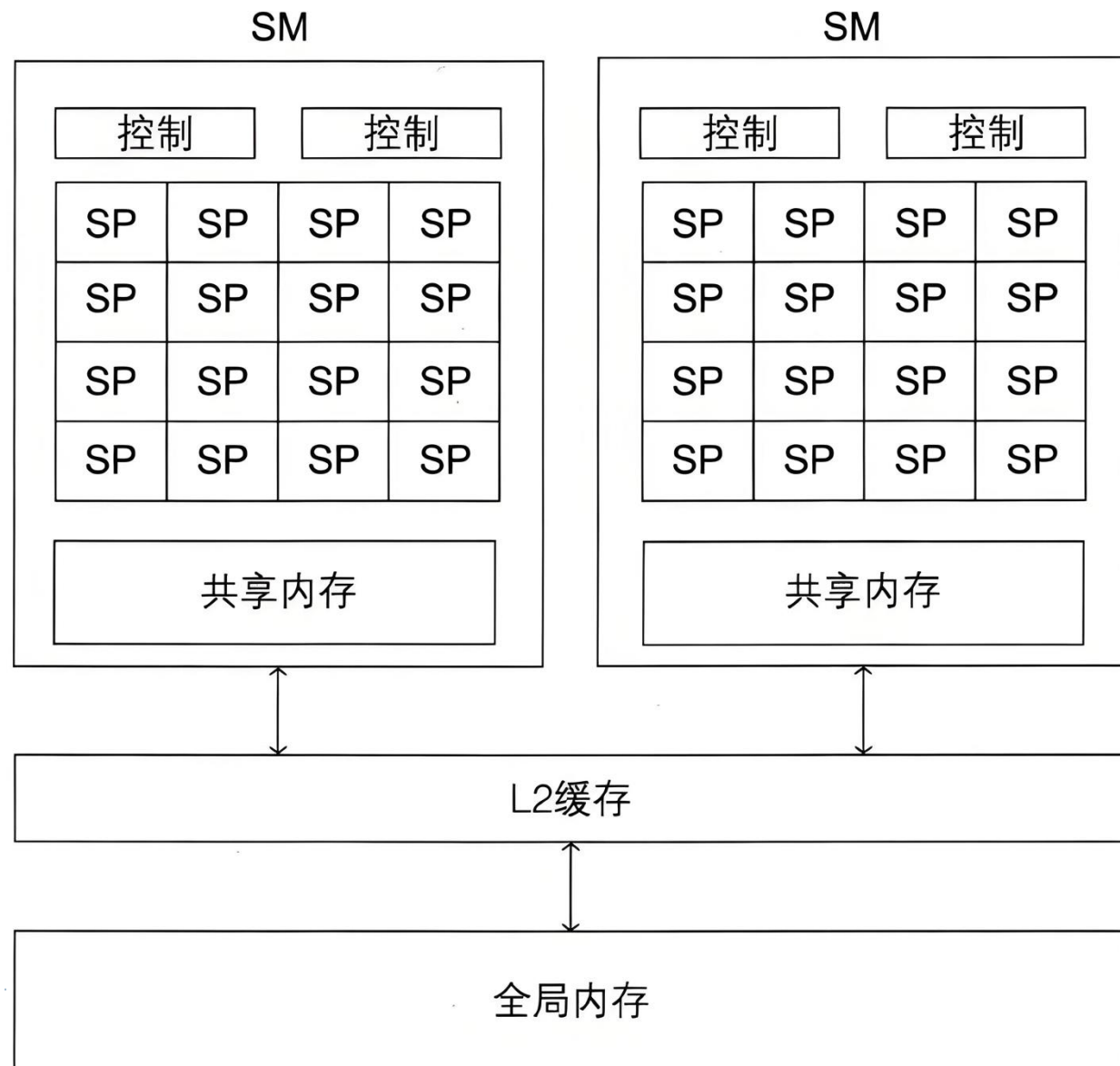
- GPU可被认为是由一个或多个SIMD处理器组成。
- SIMD处理器
 - ▶ 由一个控制单元（控制器）和多个数据通路（运算器）组成。
 - ▶ 控制单元从内存中获取指令并将其广播给数据通路。
 - ▶ 每条数据通路要么在其数据上执行指令，要么处于空闲状态。

时间	$x[i] \geq 0$ 的数据通路	$x[i] < 0$ 的数据通路
1	测试 $x[i] \geq 0$	测试 $x[i] \geq 0$
2	$x[i] += 1$	空闲
3	空闲	$x[i] -= 2$

```
if (x[i] >= 0)
    x[i] += 1;
else
    x[i] -= 2;
```

GPU架构

- NVIDIA GPU由流式多处理器（SM）组成。
- 一个SM可以有多个控制单元和更多数据通路。
 - ▶ 可将SM视为由一个或多个SIMD处理器组成。
 - ▶ 数据通路被称为核心或流处理器（SP）。
- NVIDIA GeForce RTX 4090D
 - ▶ 有114个SM。
 - ▶ 每个SM 有128个SP。
 - ▶ 总共有14592个SP。



GPU架构

- 严格意义上，是SIMT而非SIMD
 - SIMT代表单指令多线程
 - SM上执行相同指令的线程可能不会同时执行
- 主机
 - CPU及其相关内存
- 设备
 - GPU及其相关内存
- 主机和设备之间的数据传输

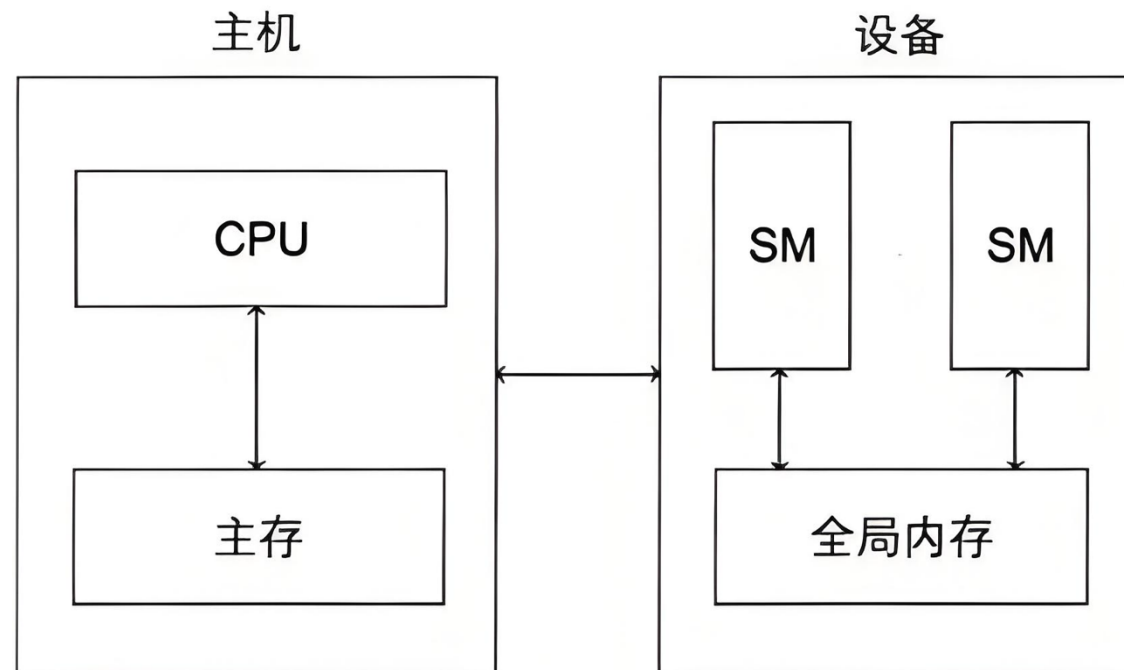
异构计算

■ 同构计算

- ▶ 具有相同架构处理器的系统。

■ 异构计算

- ▶ 具有多种架构处理器的系统。
- ▶ GPGPU计算
 - 同时使用了主机处理器（传统 CPU）和图形处理器（GPU）。
- ▶ 其他处理器：
 - 可编程门阵列（FPGA）
 - 数字信号处理器（DSP）



CPU 和 GPU 的简化框图

CUDA hello

- Compute Unified Device Architecture。
- 适用于配备NVIDIA的GPU的异构系统。
 - 。
- C、C++、FORTRAN、Python 和Java。
 - 。
- 安装CUDA Toolkit。
- 编译
 - ▶ `nvcc -o cuda_hello cuda_hello.cu`
- 运行
 - ▶ `./cuda_hello`

```
#include <stdio.h>
#include <cuda.h>
//设备代码：运行在GPU上
__global__ void Hello(void) {
    printf("Hello from thread %d!\n",
        threadIdx.x);
}
//主机代码：运行在CPU上
int main(void) {
    int thread_count = 10;
    Hello <<<1, thread_count>>> ();
    cudaDeviceSynchronize();
    return 0;
}
```

深入了解

- 执行从main函数开始，它在主机上运行。
- 对Hello函数的调用启动了核函数。
- 调用中的<<<1, thread_count>>>参数，指定了在设备上启动thread_count个核函数副本。
- 当核函数启动时，系统会初始化threadIdx结构体。
 - ▶ threadIdx.x包含线程的索引或序列号。
- 每个线程打印其信息并终止。
- cudaDeviceSynchronize迫使主机等待，直到所有的线程都完成了核函数的执行。

线程、线程块和线程网格

- NVIDIA GPU由一组SM组成，每个SM由一组SP组成。
- 每个单独的线程将在一个SP上执行其代码。
- 线程块是在单个SM上运行的线程集合。

线程、线程块和线程网格

- 在核函数调用中，尖括号中的
 - ▶ 第一个值指定线程块的数量。
 - ▶ 第二个值是每个线程块中的线程数。
- `Hello<<<1, n>>>()`
 - ▶ 启动1个线程块，其中有n个线程。
 - ▶ 使用了1个SM。
- `Hello<<<m, n>>>()`
 - ▶ 启动m个线程块，每个线程块都包含n个线程。
 - ▶ 使用了m个SM。

```
#include <stdio.h>
#include <cuda.h>

__global__ void Hello(void) {
    printf("Hello from thread %d in block %d\n", threadIdx.x, blockIdx.x);
}

int main(void) {
    int blk_ct = 2;
    int th_per_blk = 4;
    Hello <<<blk_ct, th_per_blk>>> ();
    cudaDeviceSynchronize();
    return 0;
}
```

线程、线程块和线程网格

- 网格是由核函数启动的线程块的集合。
 - ▶ 一个线程块是由线程组成的。
 - ▶ 一个网格是由线程块组成的。
- 核函数启动的三维网格信息。
 - ▶ threadIdx: 线程在其线程块中的序列号或索引。
 - ▶ blockDim: 线程块的维度、形状或大小。
 - ▶ blockIdx: 块在网格中的序列号或索引。
 - ▶ gridDim: 网格的维度、形状或大小。
- 线程块之间是独立的，可以以任何次序执行。

线程、线程块和线程网格

```
#include <stdio.h>
#include <cuda.h>
__global__ void Hello(void) {
    printf("Hello from thread(%d,%d,%d) in
    block(%d,%d,%d)\n", threadIdx.x,
    threadIdx.y, threadIdx.z, blockIdx.x,
    blockIdx.y, blockIdx.z);
}
int main(void) {
    dim3 grid_dims, block_dims;
    grid_dims.x = 1;
    grid_dims.y = 2;
    grid_dims.z = 3;
```

```
    block_dims.x = 2;
    block_dims.y = 2;
    block_dims.z = 2;
    Hello <<<grid_dims, block_dims>>> ();
    cudaDeviceSynchronize();
    return 0;
}
```

NVIDIA计算能力和设备架构

- 线程数和块数的限制。
 - 取决于NVIDIA 所定义的GPU的计算能力。
- 计算能力是一个具有a.b形式的数字。
 - a: 主要修订号, 可以是1、2、3、5、6、7、8。
 - b: 小型修订号, 取值范围为0~7。

GPU架构和计算能力

名字	Ampere	Tesla	Fermi	Kepler	Maxwell	Pascal	Volta	Turing
计算能力	8.0	1. <i>b</i>	2. <i>b</i>	3. <i>b</i>	5. <i>b</i>	6. <i>b</i>	7.0	7.5

NVIDIA计算能力和设备架构

- 分配给单个SM的最大线程数。
 - 计算能力为2.6的设备：1536。
 - 计算能力>3的设备：目前最大为2048。
- 块和网格的维度大小的限制。
 - x或y维的最大值为1024。
 - z维的最大值为64。

向量加法

```
#include <stdio.h>
#include <cuda.h>
__global__ void Vec_add(const
float x[], const float y[], float z[],
const int n) {
int my_elt = blockDim.x *
blockIdx.x + threadIdx.x;
if (my_elt < n)
z[my_elt] = x[my_elt] + y[my_elt];
}
int main(void) {
int n = 1e8;
int th_per_blk = 1024;
int blk_ct = ceil(n * 1.0 /
th_per_blk);
float* x, * y, * z, * cz;
cudaMallocManaged(&x, n *
sizeof(float));
cudaMallocManaged(&y, n *
sizeof(float));
```

```
cudaMallocManaged(&z, n *
sizeof(float));
cz = (float*)malloc(n *
sizeof(float));
for (int i = 0; i < n; i++) {
x[i] = rand() * 1.0 / RAND_MAX;
y[i] = rand() * 1.0 / RAND_MAX;
}
struct timespec start, finish;
timespec_get(&start, TIME_UTC);
Vec_add <<<blk_ct, th_per_blk>>>
(x, y, z, n);
cudaDeviceSynchronize();
timespec_get(&finish, TIME_UTC);
printf("GPU time = %f\n",
finish.tv_sec - start.tv_sec +
(finish.tv_nsec - start.tv_nsec) / 1e9);
timespec_get(&start, TIME_UTC);
for (int i = 0; i < n; i++)
cz[i] = x[i] + y[i];
```

```
timespec_get(&finish, TIME_UTC);
printf("CPU time = %f\n",
finish.tv_sec - start.tv_sec +
(finish.tv_nsec - start.tv_nsec) / 1e9);
double sum = 0;
for (int i = 0; i < n; i++) {
double diff = z[i] - cz[i];
sum += diff * diff;
}
double diff_norm = sqrt(sum);
printf("Two-norm of difference
between host and device = %f\n",
diff_norm);
cudaFree(x);
cudaFree(y);
cudaFree(z);
free(cz);
return 0;
}
```

向量加法

- $\text{blockDim.x} * \text{blockIdx.x} + \text{threadIdx.x}$
 - 为每个线程分配一个唯一的序号。
- 检查是否满足条件 $\text{my_elt} < n$
- `cudaMallocManaged`
 - 分配的内存将由“统一内存系统”自动管理。
 - 分配的内存可以在设备和主机上使用。
 - 可能比将设备内存与主机内存分开处理的核函数要慢。
- `cudaFree`
 - 释放内存。
 - 如果内存是在设备上分配的，则无法在主机上释放它，反之亦然。

显式内存传输

```
#include <stdio.h>
#include <cuda.h>
__global__ void Vec_add(const float x[],
const float y[], float z[], const int n) {
int my_elt = blockDim.x * blockIdx.x +
threadIdx.x;
if (my_elt < n)
z[my_elt] = x[my_elt] + y[my_elt];
}
int main(void) {
int n = 1e8;
int th_per_blk = 1024;
int blk_ct = ceil(n * 1.0 / th_per_blk);
float* x, * y, * z, * cz;
float* dx, * dy, * dz;
x = (float*)malloc(n * sizeof(float));
y = (float*)malloc(n * sizeof(float));
z = (float*)malloc(n * sizeof(float));
cz = (float*)malloc(n * sizeof(float));
cudaMalloc(&dx, n * sizeof(float));
cudaMalloc(&dy, n * sizeof(float));
cudaMalloc(&dz, n * sizeof(float));
```

```
for (int i = 0; i < n; i++) {
x[i] = rand() * 1.0 / RAND_MAX;
y[i] = rand() * 1.0 / RAND_MAX;
}
struct timespec start, finish;
cudaMemcpy(dx, x, n * sizeof(float),
cudaMemcpyHostToDevice);
cudaMemcpy(dy, y, n * sizeof(float),
cudaMemcpyHostToDevice);
timespec_get(&start, TIME_UTC);
Vec_add <<<blk_ct, th_per_blk>>> (dx,
dy, dz, n);
cudaDeviceSynchronize();
timespec_get(&finish, TIME_UTC);
cudaMemcpy(z, dz, n * sizeof(float),
cudaMemcpyDeviceToHost);
printf("GPU time = %fn", finish.tv_sec
- start.tv_sec + (finish.tv_nsec -
start.tv_nsec) / 1e9);
timespec_get(&start, TIME_UTC);
for (int i = 0; i < n; i++)
cz[i] = x[i] + y[i];
```

```
timespec_get(&finish, TIME_UTC);
printf("CPU time = %fn", finish.tv_sec
- start.tv_sec + (finish.tv_nsec -
start.tv_nsec) / 1e9);
double sum = 0;
for (int i = 0; i < n; i++) {
double diff = z[i] - cz[i];
sum += diff * diff;
}
double diff_norm = sqrt(sum);
printf("Two-norm of difference between
host and device = %fn", diff_norm);
free(x);
free(y);
free(z);
free(cz);
cudaFree(dx);
cudaFree(dy);
cudaFree(dz);
return 0;
}
```

显式内存传输

■ cudaMalloc

- ▶ CUDA在GPU显存上分配内存的函数。
- ▶ 专门给核函数（GPU代码）用。
- ▶ 不能直接在CPU上读写。

■ cudaMemcpy

- ▶ 负责在主机内存（CPU）和设备显存（GPU）之间拷贝数据。
- ▶ 必须指定拷贝方向：
 - cudaMemcpyHostToDevice
 - cudaMemcpyDeviceToHost

从CUDA核函数返回结果

```
#include <stdio.h>
#include <cuda.h>

__global__ void Add(int x, int y, int* sum_p) {
    *sum_p = x + y;
}

int main(void) {
    int sum = -5;
    Add <<<1, 1>>> (2, 3, &sum);
    cudaDeviceSynchronize();
    printf("The sum is %d\n", sum);
    return 0;
}
```

“统一内存系统” 分配

```
#include <stdio.h>
#include <cuda.h>
__global__ void Add(int x, int y, int*
sum_p) {
    *sum_p = x + y;
}
int main(void) {
    int* sum_p;
    cudaMallocManaged(&sum_p,
sizeof(int));
    *sum_p = -5;
    Add <<<1, 1>>> (2, 3, sum_p);
    cudaDeviceSynchronize();
```

```
printf("The sum is %d\n", *sum_p);
cudaFree(sum_p);
return 0;
}
```

从设备显式复制到主机

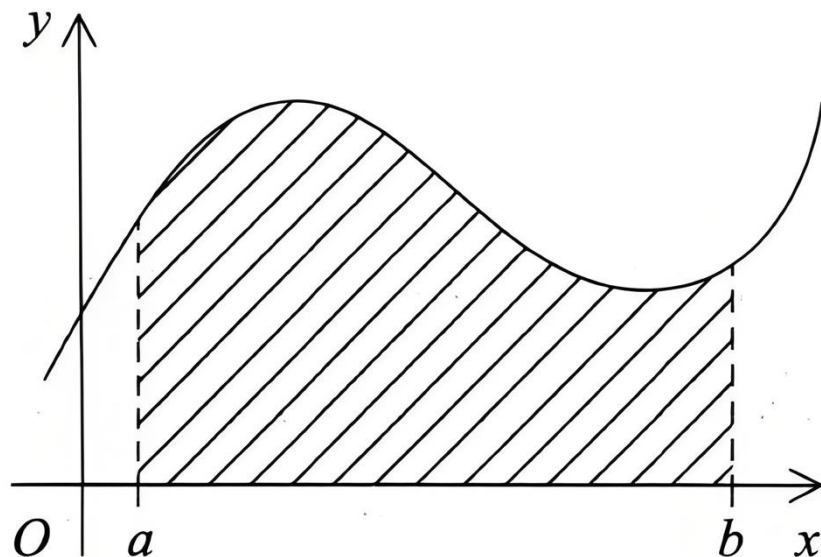
```
#include <stdio.h>
#include <cuda.h>
__global__ void Add(int x, int y, int*
sum_p) {
    *sum_p = x + y;
}
int main(void) {
    int* hsum_p, * dsum_p;
    hsum_p = (int*)malloc(sizeof(int));
    cudaMalloc(&dsum_p, sizeof(int));
    *hsum_p = -5;
    Add <<<1, 1>>> (2, 3, dsum_p);
    cudaDeviceSynchronize();
```

```
    cudaMemcpy(hsum_p, dsum_p,
sizeof(int), cudaMemcpyDeviceToHost);
    printf("The sum is %d\n", *hsum_p);
    free(hsum_p);
    cudaFree(dsum_p);
    return 0;
}
```

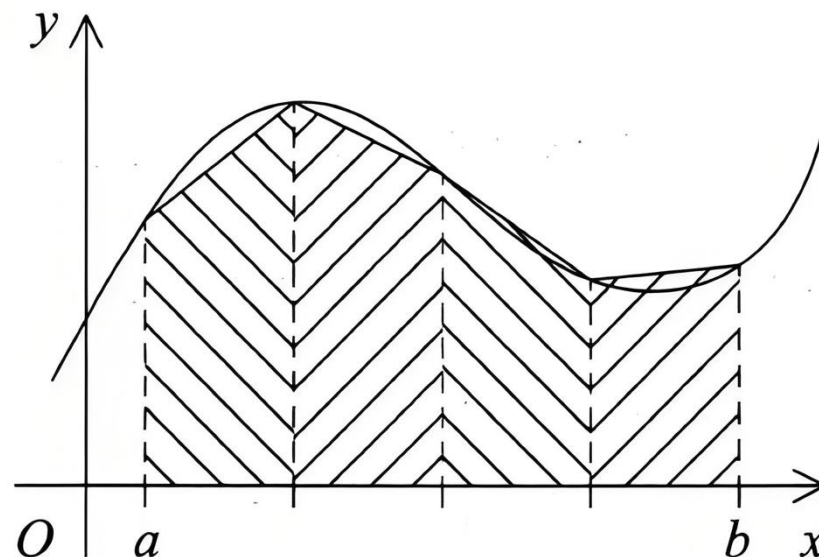
使用全局托管变量

```
#include <stdio.h>
#include <cuda.h>
__managed__ int sum;
__global__ void Add(int x, int y) {
    sum = x + y;
}
int main(void) {
    sum = -5;
    Add <<<1, 1>>> (2, 3);
    cudaDeviceSynchronize();
    printf("The sum is %d\n", sum);
    return 0;
}
```


梯形法则



拟估算面积



利用梯形近似面积

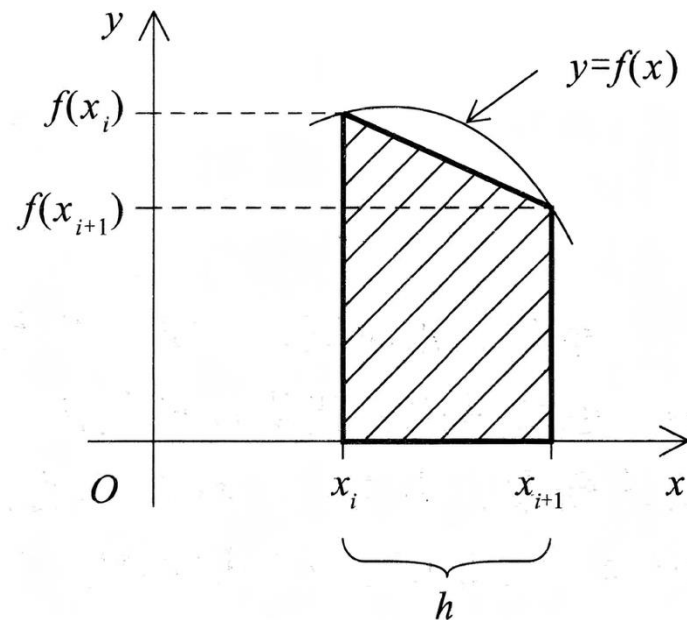
梯形法则

$$\text{梯形面积} = \frac{h}{2} [f(x_i) + f(x_{i+1})]$$

$$h = \frac{b-a}{n}$$

$$x_0 = a, x_1 = a + h, x_2 = a + 2h, \dots, x_{n-1} = a + (n-1)h, x_n = b$$

$$\text{梯形面积的总和} = h[f(x_0)/2 + f(x_1) + f(x_2) + \dots + f(x_{n-1}) + f(x_n)/2]$$



串行程序

```
#include <stdio.h>
float f(const float x) {
    return 4 / (1 + x * x);
}
float Trap(const float a, const float b,
const int n) {
    float x, h = (b - a) / n;
    float trap = 0.5 * (f(a) + f(b));
    for (int i = 1; i <= n - 1; i++) {
        x = a + i * h;
        trap += f(x);
    }
    return trap * h;
}
```

```
}
int main(void) {
    float integral;
    float a = 0.0, b = 1.0;
    int n = 10000;
    integral = Trap(a, b, n);
    printf("our estimate of the integral =
%f\n", integral);
    return 0;
}
```

CUDA梯形法则I

```
#include <stdio.h>
#include <cuda.h>
__host__ __device__ float f(float x)
{
    return 4 / (1 + x * x);
}
__global__ void Dev_trap(const
float a, const float b, const float h,
const int n, float* trap_p) {
    int my_i = blockDim.x * blockIdx.x
+ threadIdx.x;
    if (0 < my_i && my_i < n) {
        float my_x = a + my_i * h;
        float my_trap = f(my_x);
        atomicAdd(trap_p, my_trap);
    }
}
int main(void) {
    int n = 1 << 20;
```

```
int th_per_blk = 1024;
int blk_ct = ceil(n * 1.0 /
th_per_blk);
float a = 0.0, b = 1.0, x, * trap_p,
trap;
cudaMallocManaged(&trap_p,
sizeof(float));
struct timespec start, finish;
timespec_get(&start, TIME_UTC);
*trap_p = 0.5 * (f(a) + f(b));
float h = (b - a) / n;
Dev_trap <<<blk_ct, th_per_blk>>>
(a, b, h, n, trap_p);
cudaDeviceSynchronize();
*trap_p = h * (*trap_p);
timespec_get(&finish, TIME_UTC);
printf("GPU time = %f\n",
finish.tv_sec - start.tv_sec +
(finish.tv_nsec - start.tv_nsec) / 1e9);
```

```
timespec_get(&start, TIME_UTC);
trap = 0.5 * (f(a) + f(b));
for (int i = 1; i <= n - 1; i++) {
    x = a + i * h;
    trap += f(x);
}
trap = trap * h;
timespec_get(&finish, TIME_UTC);
printf("CPU time = %f\n",
finish.tv_sec - start.tv_sec +
(finish.tv_nsec - start.tv_nsec) / 1e9);
printf("The area as computed by
cuda is: %f\n", *trap_p);
printf("The area as computed by cpu
is: %f\n", trap);
cudaFree(trap_p);
return 0;
}
```

CUDA梯形法则I

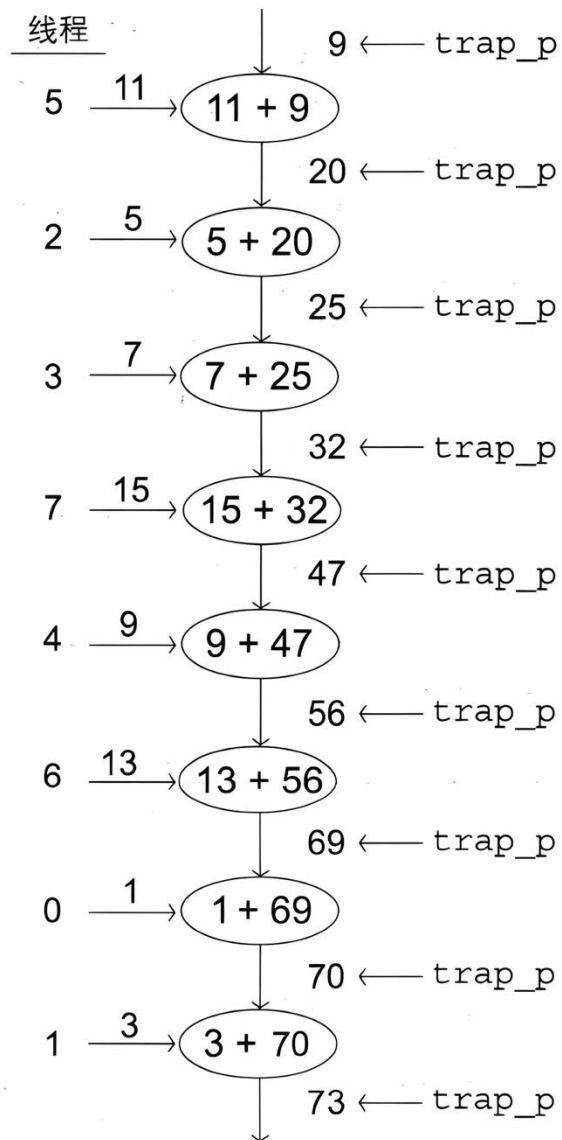
- 核函数和函数参数对执行中的线程是私有的。
- 形参是在线程的堆栈上分配的。
- *trap_p是共享变量。
 - atomicAdd
 - 原子操作

系统	ARM Cortex-A15	NVIDIA GK20A	Intel Core i7	NVIDIA GeForce GTX Titan X
时钟	2.3GHz	852MHz	3.5GHz	1.08GHz
SM的数量, SP的数量	—	1 192	—	243 072
运行时间	33.6	20.7	4.48	3.08

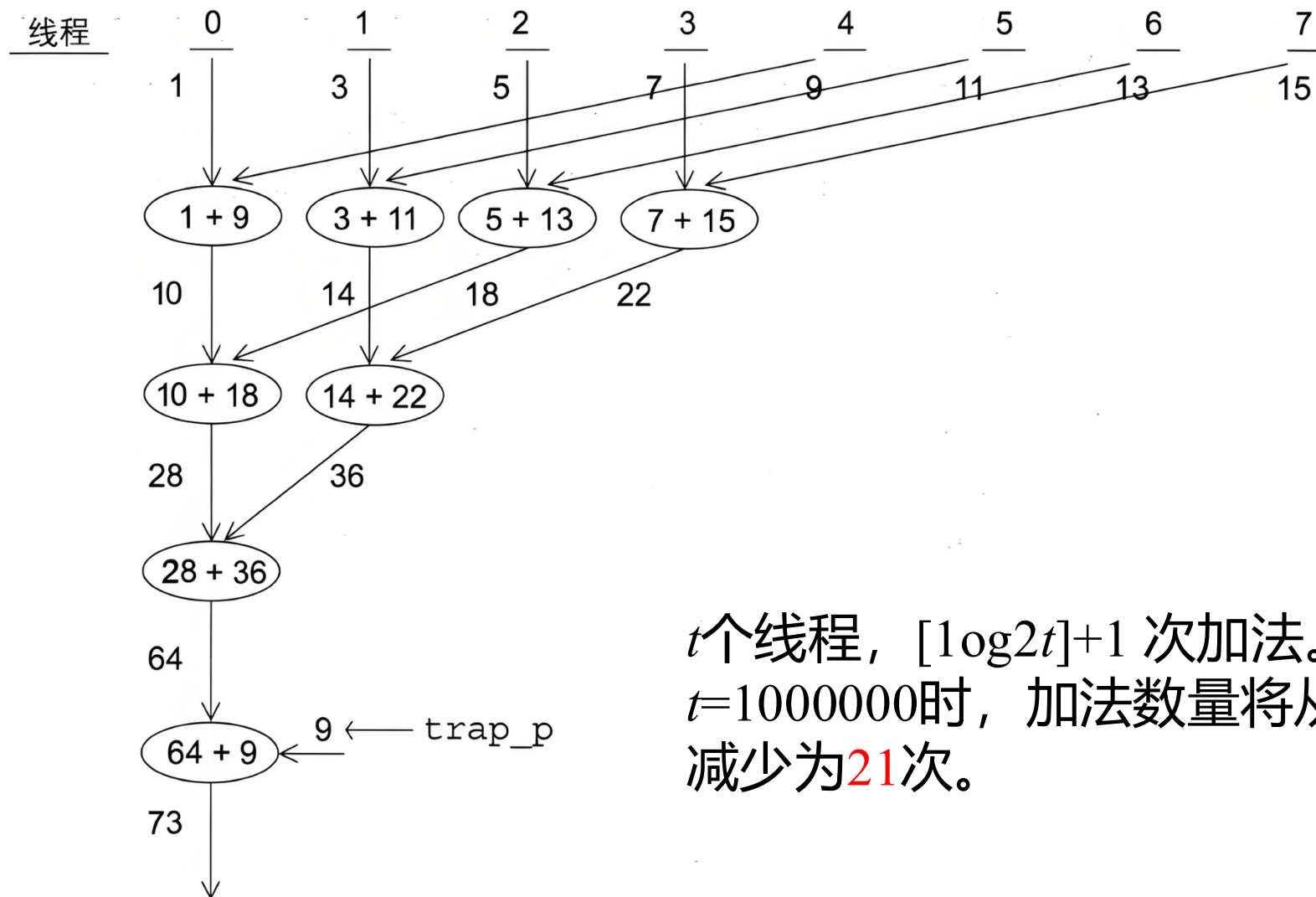
CUDA梯形法则II: 提升性能

■ atomicAdd的调用会串行化线程。

时间	线程	my_trap	*trap_p
Start	—	—	9
t_0	5	11	20
t_1	2	5	25
t_2	3	7	32
t_3	7	15	47
t_4	4	9	56
t_5	6	13	69
t_6	0	1	70
t_7	1	3	73



树形通信



t 个线程, $\lceil \log_2 t \rceil + 1$ 次加法。
 $t=1000000$ 时, 加法数量将从1000000次
减少为21次。

树结构求和的两种标准实现

- 共享内存
 - 适用于计算能力 <3 的设备。
- 线程束洗牌函数
 - 允许一个束中的一组线程读取该束中其他线程存储的变量。

GPU内存层次结构

■ 全局内存

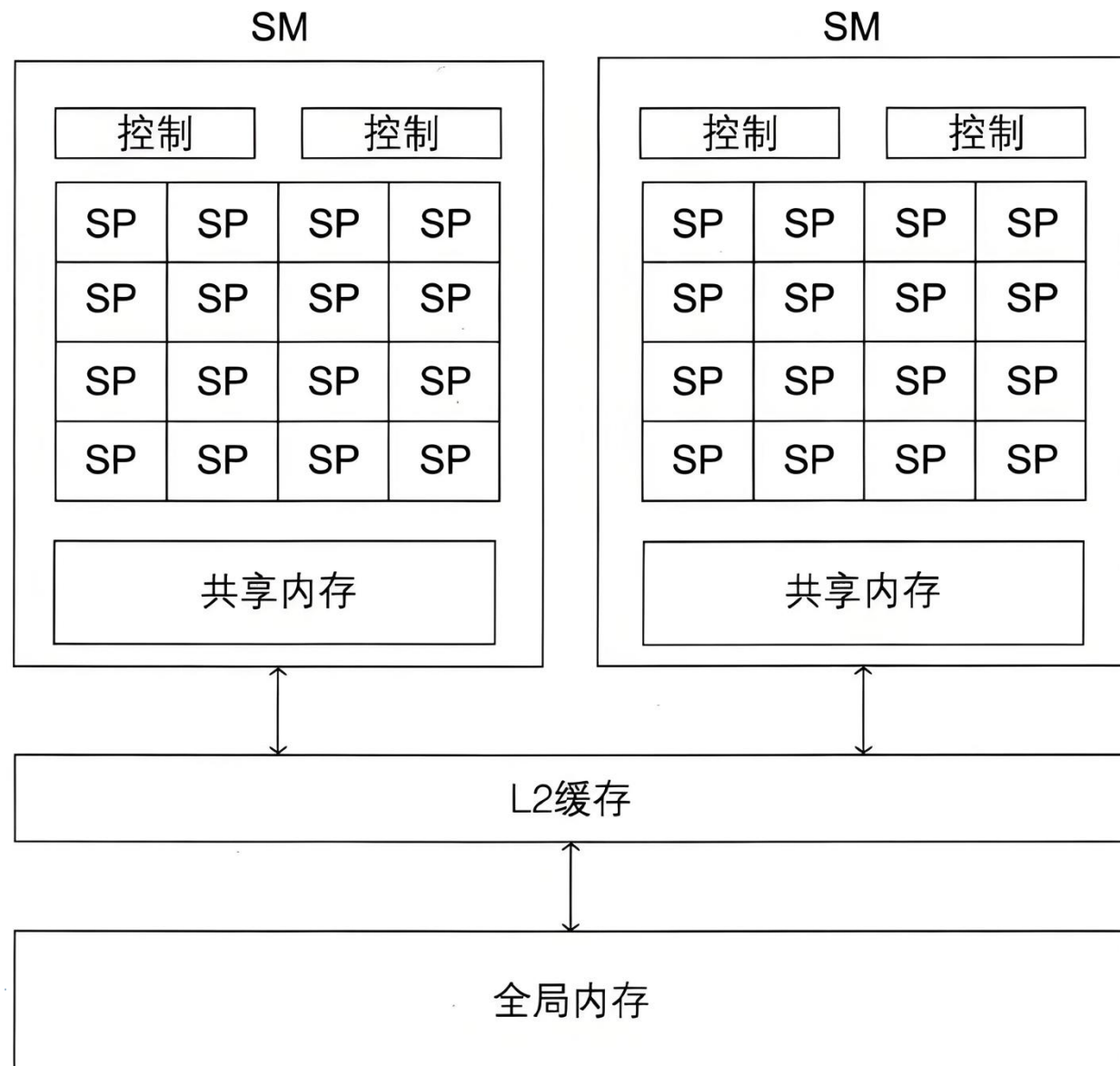
- ▶ 在底部，最慢、最大。
- ▶ 所有SP和所有线程都可以访问。

■ 共享内存

- ▶ 在中间，更快、更小。
- ▶ 只能由属于该SM的SP访问。

■ 寄存器

- ▶ 在顶部，最快、最小。



提高核函数的性能

- 增加对寄存器的使用。
- 减少对共享和 / 或全局内存的使用。

一些NVIDIA GPU的内存统计

GPU	计算能力	寄存器：字节每线程	共享内存：字节每块	全局内存：字节每 GPU
Quadro 600	2.1	504	48K	1G
GK20A (Jetson TK1)	3.2	504	48K	2G
GeForce GTX Titan X	5.2	504	48K	12G

线程束

- 属于一个线程块的一组线程，具有连续序列号。
- 大小是固定的，为32个线程。
 - 存于系统变量warpSize中。
- GPU调度和执行的基本单位。
- 以SIMD方式运行，同一线程束内的线程会同时执行相同的指令。
- 线程块的大小设置为32的倍数，以充分利用并行性。

线程块的大小

```
#include <stdio.h>
#include <cuda.h>
__global__ void Vec_add(const float x[],
const float y[], float z[], const int n) {
int my_elt = blockDim.x * blockIdx.x +
threadIdx.x;
if (my_elt < n)
z[my_elt] = x[my_elt] + y[my_elt];
}
int main(void) {
int n = 1e8;
int th_per_blk = 1024;
int blk_ct = ceil(n * 1.0 / th_per_blk);
float* x, * y, * z, * cz;
float* dx, * dy, * dz;
x = (float*)malloc(n * sizeof(float));
y = (float*)malloc(n * sizeof(float));
z = (float*)malloc(n * sizeof(float));
cz = (float*)malloc(n * sizeof(float));
cudaMalloc(&dx, n * sizeof(float));
cudaMalloc(&dy, n * sizeof(float));
cudaMalloc(&dz, n * sizeof(float));
```

```
for (int i = 0; i < n; i++) {
x[i] = rand() * 1.0 / RAND_MAX;
y[i] = rand() * 1.0 / RAND_MAX;
}
struct timespec start, finish;
cudaMemcpy(dx, x, n * sizeof(float),
cudaMemcpyHostToDevice);
cudaMemcpy(dy, y, n * sizeof(float),
cudaMemcpyHostToDevice);
timespec_get(&start, TIME_UTC);
Vec_add <<<blk_ct, th_per_blk>>> (dx,
dy, dz, n);
cudaDeviceSynchronize();
timespec_get(&finish, TIME_UTC);
cudaMemcpy(z, dz, n * sizeof(float),
cudaMemcpyDeviceToHost);
printf("GPU time = %fn", finish.tv_sec
- start.tv_sec + (finish.tv_nsec -
start.tv_nsec) / 1e9);
timespec_get(&start, TIME_UTC);
for (int i = 0; i < n; i++)
cz[i] = x[i] + y[i];
```

```
timespec_get(&finish, TIME_UTC);
printf("CPU time = %fn", finish.tv_sec
- start.tv_sec + (finish.tv_nsec -
start.tv_nsec) / 1e9);
double sum = 0;
for (int i = 0; i < n; i++) {
double diff = z[i] - cz[i];
sum += diff * diff;
}
double diff_norm = sqrt(sum);
printf("Two-norm of difference between
host and device = %fn", diff_norm);
free(x);
free(y);
free(z);
free(cz);
cudaFree(dx);
cudaFree(dy);
cudaFree(dz);
return 0;
}
```

线程束

■ 分歧

- ▶ 一个线程束中的线程试图执行不同的语句，例如在if-else语句中采用不同的分支。

■ 收敛

- ▶ 当分歧线程完成不同语句的执行，并开始执行相同的语句。

```
if (x[i] >= 0)
    x[i] += 1;
else
    x[i] -= 2;
```

■ 通道

- ▶ 线程束中线程的序列号。
- ▶ `lane = threadIdx.x % warpSize`

线程束洗牌函数

- `__device__ float __shfl_down_sync(unsigned mask, float var, unsigned diff, int width = warpSize)`
- 掩码(mask) 参数指示哪些线程正在参与调用。
 - mask的第*i*位指示通道号为*i*的线程是否参与。
 - 通常会使用线程束中的所有线程，即mask = 0xffffffff
- 如果通道号为*l*的线程调用__shfl_down_sync, 那么通道号为*l*+diff的线程上存储在var中的值被返回给它。
- 向下洗牌
 - diff 的类型是无符号，返回的值来自更高序列号的线程。

线程束洗牌函数

■ 为了避免不明确的结果

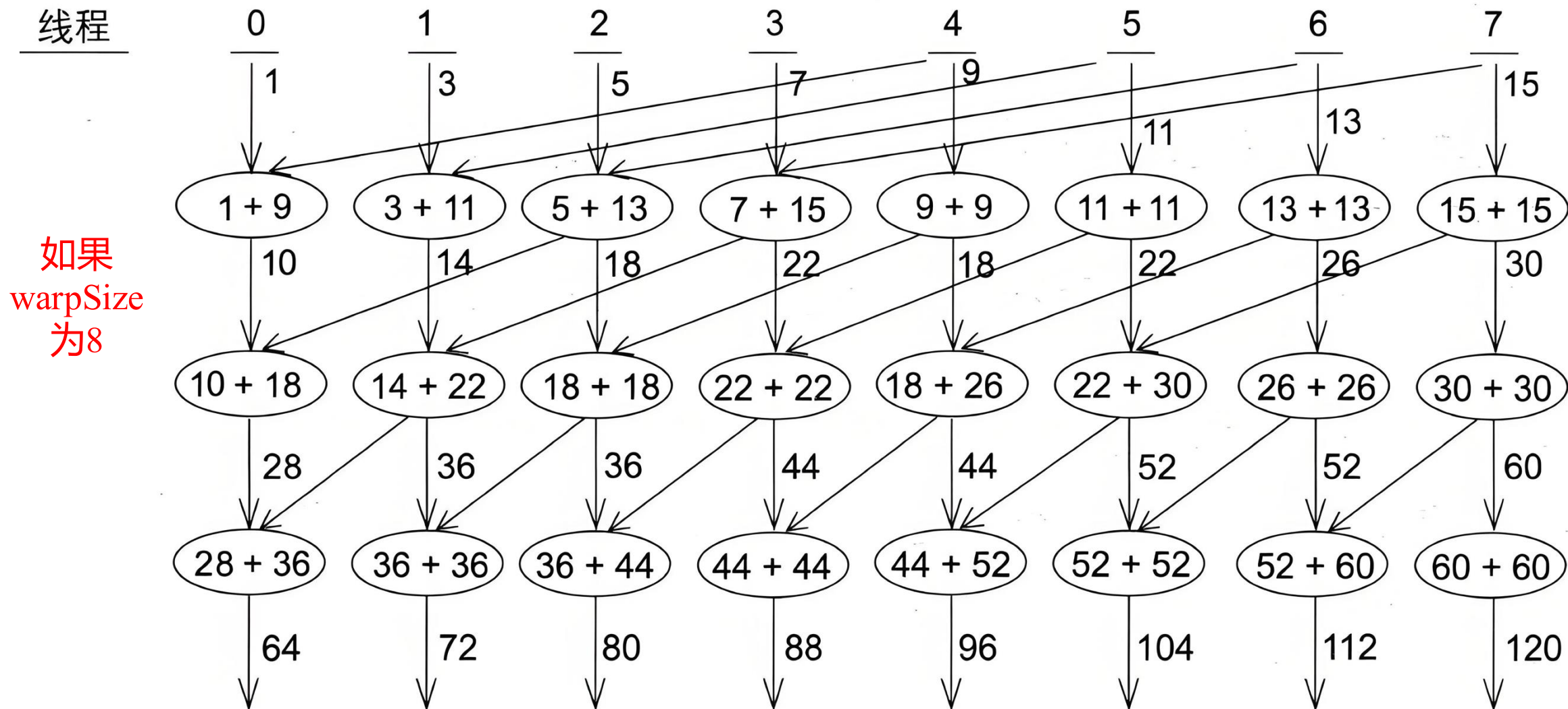
- ▶ 线程束中的所有线程调用__shfl_down_sync。
- ▶ 所有的线程束都有warpSize个线程，即线程块大小是warpSize的倍数。

线程束洗牌函数

```
#include <stdio.h>
#include <cuda.h>
__global__ void Shuffle(int x[]) {
    int my_elt = threadIdx.x;
    x[my_elt] = __shfl_down_sync(0xffffffff,
    x[my_elt], 1);
}
int main(void) {
    int n = 40;
    int* x;
    cudaMallocManaged(&x, n * sizeof(int));
    for (int i = 0; i < n; i++)
        x[i] = i;
```

```
Shuffle <<<1, n>>> (x);
cudaDeviceSynchronize();
for (int i = 0; i < n; i++)
    printf("%d:\t%d\n", i, x[i]);
    cudaFree(x);
    return 0;
}
```


使用线程束洗牌的树形求和



使用线程束洗牌实现树形全局求和

```
#include <stdio.h>
#include <cuda.h>

__device__ float warp_sum(float var) {
    for (int diff = warpSize / 2; diff > 0; diff =
        diff / 2)
        var += __shfl_down_sync(0xffffffff, var,
            diff);
    return var;
}

__global__ void Sum(float* sum_p) {
    float var = threadIdx.x;
    float result = warp_sum(var);
    if (threadIdx.x == 0)
```

```
        *sum_p = result;
    }
    int main(void) {
        float* sum_p;
        cudaMallocManaged(&sum_p,
            sizeof(float));
        Sum <<<1, 32>>> (sum_p);
        cudaDeviceSynchronize();
        printf("%f\n", *sum_p);
        cudaFree(sum_p);
        return 0;
    }
```

线程束洗牌的替代方案

■ 如果GPU计算能力<3.0

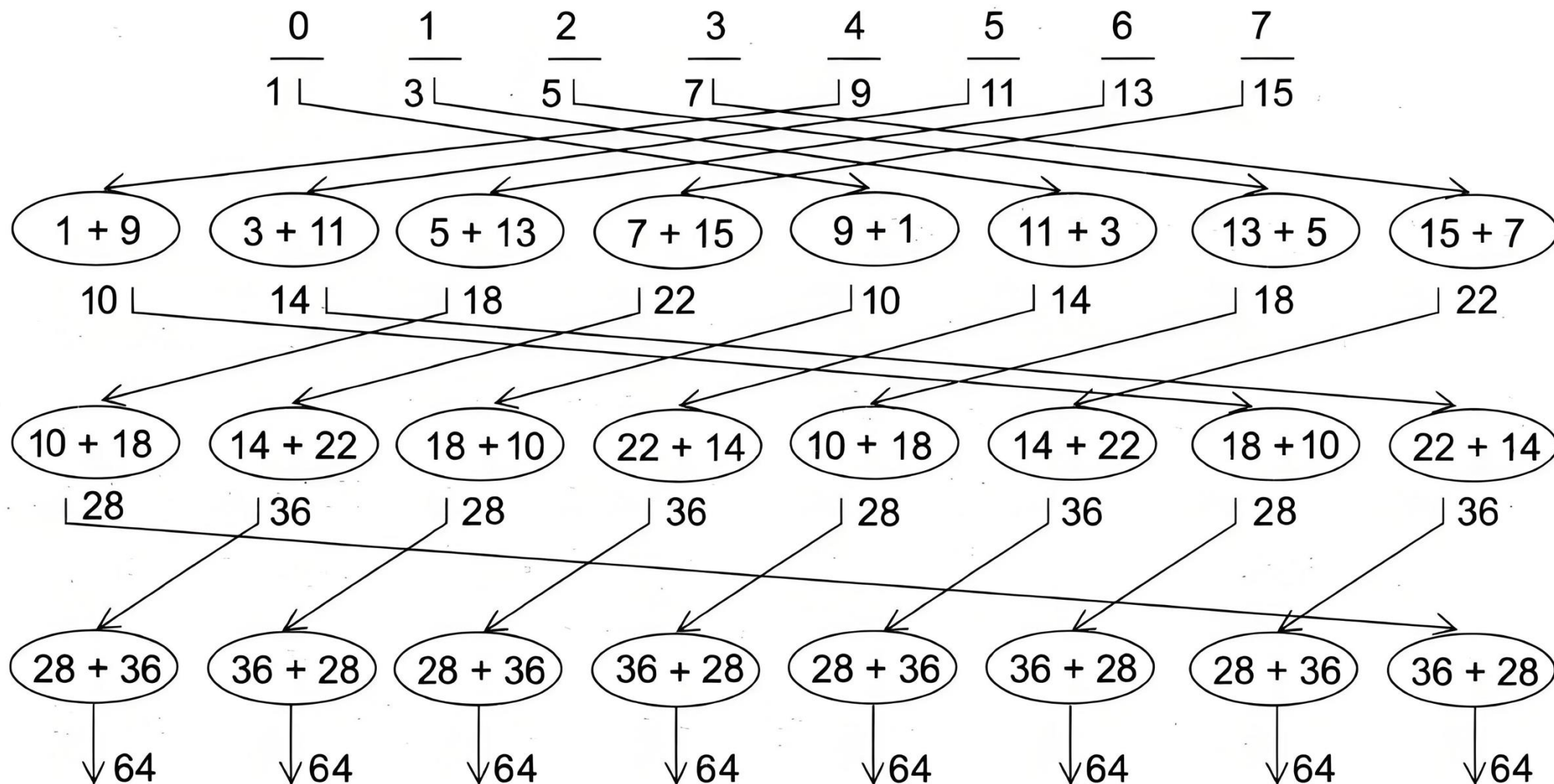
- ▶ 无法使用线程束洗牌函数。
- ▶ 一个线程将无法直接访问其他线程的寄存器。

■ 共享内存

- ▶ 同一线程块中的线程都可以访问相同的共享内存位置。
- ▶ 比寄存器访问慢。

使用共享内存的传播求和

线程



使用共享内存的传播求和

```
#include <stdio.h>
#include <cuda.h>

__device__ float Shared_mem_sum(float
shared_vals[]) {
    int my_lane = threadIdx.x % warpSize;
    for (int diff = warpSize / 2; diff > 0; diff = diff
/ 2) {
        int source = (my_lane + diff) % warpSize;
        shared_vals[my_lane] += shared_vals[source];
    }
    return shared_vals[my_lane];
}

__global__ void Sum(float* sum_p) {
    __shared__ float shared_vals[32];
    shared_vals[threadIdx.x] = threadIdx.x;
```

```
float result = Shared_mem_sum(shared_vals);
if (threadIdx.x == 0)
    *sum_p = result;
}

int main(void) {
    float* sum_p;
    cudaMallocManaged(&sum_p, sizeof(float));
    Sum <<<1, 32>>> (sum_p);
    cudaDeviceSynchronize();
    printf("%f\n", *sum_p);
    cudaFree(sum_p);
    return 0;
}
```

用大小为warpSize的线程块实现梯形法则

- 线程块由warpSize个线程组成。
- 使用“树形”求和将线程束中线程的结果加起来。
- 线程束中通道ID为0的线程将使用atomicAdd将线程束的总和累加到总数中。

使用线程束洗牌

```
#include <stdio.h>
#include <cuda.h>
__host__ __device__ float f(float x) {
    return 4 / (1 + x * x);
}
__device__ float Warp_sum(const float
my_val) {
    unsigned mask = 0xffffffff;
    float result = my_val;
    for (unsigned diff = warpSize / 2; diff > 0;
        diff = diff / 2)
        result += __shfl_down_sync(mask, result,
diff);
    return result;
}
__global__ void Dev_trap(const float a,
const float b, const float h, const int n,
float* trap_p) {
    int my_i = blockDim.x * blockIdx.x +
threadIdx.x;
    float my_trap = 0.0;
    if (0 < my_i && my_i < n) {
```

```
float my_x = a + my_i * h;
my_trap = f(my_x);
}
float result = Warp_sum(my_trap);
if (threadIdx.x % warpSize == 0)
    atomicAdd(trap_p, result);
}
int main(void) {
    int n = 1 << 20;
    int th_per_blk = 1024;
    int blk_ct = ceil(n * 1.0 / th_per_blk);
    float a = 0.0, b = 1.0, x, * trap_p, trap;
    cudaMallocManaged(&trap_p,
sizeof(float));
    struct timespec start, finish;
    timespec_get(&start, TIME_UTC);
    *trap_p = 0.5 * (f(a) + f(b));
    float h = (b - a) / n;
    Dev_trap <<<blk_ct, th_per_blk>>> (a, b,
h, n, trap_p);
    cudaDeviceSynchronize();
    *trap_p = h * (*trap_p);
```

```
timespec_get(&finish, TIME_UTC);
printf("GPU time = %f\n", finish.tv_sec -
start.tv_sec + (finish.tv_nsec - start.tv_nsec)
/ 1e9);
timespec_get(&start, TIME_UTC);
trap = 0.5 * (f(a) + f(b));
for (int i = 1; i <= n - 1; i++) {
    x = a + i * h;
    trap += f(x);
}
trap = trap * h;
timespec_get(&finish, TIME_UTC);
printf("CPU time = %f\n", finish.tv_sec -
start.tv_sec + (finish.tv_nsec - start.tv_nsec)
/ 1e9);
printf("The area as computed by cuda is:
%f\n", *trap_p);
printf("The area as computed by cpu is:
%f\n", trap);
cudaFree(trap_p);
return 0;
}
```


使用共享内存

```
#include <stdio.h>
#include <cuda.h>
__host__ __device__ float f(float x) {
    return 4 / (1 + x * x);
}
__device__ float Shared_mem_sum(float
shared_vals[]){
    int my_lane = threadIdx.x % warpSize;
    for (unsigned diff = 16; diff > 0; diff >>= 1)
    {
        int source = (my_lane + diff) % warpSize;
        shared_vals[my_lane] +=
        shared_vals[source];
    }
    return shared_vals[my_lane];
}
__global__ void Dev_trap(const float a,
const float b, const float h, const int n,
float* trap_p) {
    __shared__ float shared_vals[32];
    int my_i = blockDim.x * blockIdx.x +
    threadIdx.x;
    int my_lane = threadIdx.x % warpSize;
    shared_vals[my_lane] = 0.0;
```

```
if (0 < my_i && my_i < n) {
    float my_x = a + my_i * h;
    shared_vals[my_lane] = f(my_x);
}
float result =
Shared_mem_sum(shared_vals);
if (threadIdx.x % warpSize == 0)
    atomicAdd(trap_p, result);
}
int main(void) {
    int n = 1 << 20;
    int th_per_blk = 32;
    int blk_ct = ceil(n * 1.0 / th_per_blk);
    float a = 0.0, b = 1.0, x, * trap_p, trap;
    cudaMallocManaged(&trap_p,
sizeof(float));
    struct timespec start, finish;
    timespec_get(&start, TIME_UTC);
    *trap_p = 0.5 * (f(a) + f(b));
    float h = (b - a) / n;
    Dev_trap <<<blk_ct, th_per_blk>>> (a, b,
h, n, trap_p);
    cudaDeviceSynchronize();
    *trap_p = h * (*trap_p);
```

```
timespec_get(&finish, TIME_UTC);
printf("GPU time = %f\n", finish.tv_sec -
start.tv_sec + (finish.tv_nsec - start.tv_nsec)
/ 1e9);
timespec_get(&start, TIME_UTC);
trap = 0.5 * (f(a) + f(b));
for (int i = 1; i <= n - 1; i++) {
    x = a + i * h;
    trap += f(x);
}
trap = trap * h;
timespec_get(&finish, TIME_UTC);
printf("CPU time = %f\n", finish.tv_sec -
start.tv_sec + (finish.tv_nsec - start.tv_nsec)
/ 1e9);
printf("The area as computed by cuda is:
%f\n", *trap_p);
printf("The area as computed by cpu is:
%f\n", trap);
cudaFree(trap_p);
return 0;
}
```


性能

■ 使用32个线程块的梯形法则的平均运行时间。

系统	ARM Cortex-A15	NVIDIA GK20A	Intel Core i7	NVIDIA GeForce GTX Titan X
时钟	2.3 GHz	852 MHz	3.5 GHz	1.08 GHz
SM 的数量, SP 的数量		1, 192		24, 3 072
原始的	33.6	20.7	4.48	3.08
线程束洗牌		14.4		0.210
共享内存		15.0		0.206

CUDA梯形法则III: 使用具有多个线程束的线程块

■ 步骤

- ▶ (1)每个线程计算它的贡献值。
- ▶ (2)每个线程束求束内所有线程的贡献值之和。
- ▶ (3)线程束0求块中所有线程束的贡献值之和。

■ 问题

- ▶ 竞争
 - 步骤(2)必须在步骤(3)之前完成。
- ▶ 快速栅栏
 - `__device__ void __syncthreads(void)`
 - 块内线程等待，直到块中所有线程都开始执行调用。

避免竞争

- (1)每个线程计算它的贡献值。
- (2)每个线程束求束内所有线程的贡献值之和。
- (3)__syncthreads()
- (4)线程束0求块中所有线程束的贡献值之和。

避免竞争

■ __syncthreads

- ▶ 确保块中的所有线程都执行调用。
- ▶ 只同步一个块中的线程。

```
int my_x = threadIdx.x;  
if (my_x < blockDim.x / 2)  
    __syncthreads();  
my_x++;
```

线程束0如何访问其他线程束的贡献值

- 不能使用线程束洗牌和寄存器。
 - ▶ 线程束洗牌只允许线程读取束内其他线程的寄存器。
- 使用共享内存。
 - ▶ 线程束 ω 中通道号为0的线程可以将线程束的贡献值存储在warp_sum_arr[ω]中。
 - ▶ 对线程束0使用线程束洗牌求和。

使用共享内存求线程束总和

```
#include <stdio.h>
#include <cuda.h>
__host__ __device__ float f(float x) {
    return 4 / (1 + x * x);
}
__device__ float Warp_sum(const float
my_val) {
    unsigned mask = 0xffffffff;
    float result = my_val;
    for (unsigned diff = warpSize / 2; diff > 0; diff
= diff / 2)
        result += __shfl_down_sync(mask, result, diff);
    return result;
}
__global__ void Dev_trap(const float a, const
float b, const float h, const int n, float* trap_p)
{
    __shared__ float warp_sum_arr[32];
    for (int i = threadIdx.x; i < 32; i +=
blockDim.x)
        warp_sum_arr[i] = 0;
    __syncthreads();
    int my_i = blockDim.x * blockIdx.x +
threadIdx.x;
    int my_warp = threadIdx.x / warpSize;
    int my_lane = threadIdx.x % warpSize;
```

```
float blk_result = 0.0;
float my_trap = 0.0;
if (0 < my_i && my_i < n) {
    float my_x = a + my_i * h;
    my_trap = f(my_x);
}
float my_result = Warp_sum(my_trap);
if (my_lane == 0)
    warp_sum_arr[my_warp] = my_result;
__syncthreads();
if (my_warp == 0)
    blk_result =
Warp_sum(warp_sum_arr[threadIdx.x]);
if (threadIdx.x == 0)
    atomicAdd(trap_p, blk_result);
}
int main(void) {
    int n = 1 << 20;
    int th_per_blk = 1024;
    int blk_ct = ceil(n * 1.0 / th_per_blk);
    float a = 0.0, b = 1.0, x, * trap_p, trap;
    cudaMallocManaged(&trap_p, sizeof(float));
    struct timespec start, finish;
    timespec_get(&start, TIME_UTC);
    *trap_p = 0.5 * (f(a) + f(b));
    float h = (b - a) / n;
```

```
Dev_trap <<< blk_ct, th_per_blk >>> (a, b, h,
n, trap_p);
cudaDeviceSynchronize();
*trap_p = h * (*trap_p);
timespec_get(&finish, TIME_UTC);
printf("GPU time = %f\n", finish.tv_sec -
start.tv_sec + (finish.tv_nsec - start.tv_nsec) /
1e9);
timespec_get(&start, TIME_UTC);
trap = 0.5 * (f(a) + f(b));
for (int i = 1; i <= n - 1; i++) {
    x = a + i * h;
    trap += f(x);
}
trap = trap * h;
timespec_get(&finish, TIME_UTC);
printf("CPU time = %f\n", finish.tv_sec -
start.tv_sec + (finish.tv_nsec - start.tv_nsec) /
1e9);
printf("The area as computed by cuda is:
%f\n", *trap_p);
printf("The area as computed by cpu is: %f\n",
trap);
cudaFree(trap_p);
return 0;
}
```

使用共享内存的线程束求和

- 共享变量由块中所有线程共享。
- `thread_calcs`数组保存每个线程的贡献值。
- 每个线程束都通过使用共享内存来求束内所有线程的贡献值之和。

使用共享内存的线程束求和

```
#include <stdio.h>
#include <cuda.h>
__host__ __device__ float f(float x) {
    return 4 / (1 + x * x);
}
__device__ float Shared_mem_sum(float
shared_vals[]) {
    int my_lane = threadIdx.x % warpSize;
    for (unsigned diff = 16; diff > 0; diff >>= 1)
    {
        int source = (my_lane + diff) % warpSize;
        shared_vals[my_lane] += shared_vals[source];
    }
    return shared_vals[my_lane];
}
__global__ void Dev_trap(const float a, const
float b, const float h, const int n, float* trap_p) {
    __shared__ float thread_calcs[1024];
    __shared__ float warp_sum_arr[32];
    for (int i = threadIdx.x; i < 1024; i += blockDim.x)
        thread_calcs[i] = 0;
    for (int i = threadIdx.x; i < 32; i += blockDim.x)
        warp_sum_arr[i] = 0;
    __syncthreads();
    int my_i = blockDim.x * blockIdx.x + threadIdx.x;
    int my_warp = threadIdx.x / warpSize;
```

```
int my_lane = threadIdx.x % warpSize;
float blk_result = 0.0;
if (0 < my_i && my_i < n) {
    float my_x = a + my_i * h;
    thread_calcs[threadIdx.x] = f(my_x);
}
float my_result = Shared_mem_sum(thread_calcs
+ my_warp * warpSize);
if (my_lane == 0)
    warp_sum_arr[my_warp] = my_result;
__syncthreads();
if (my_warp == 0)
    blk_result = Shared_mem_sum(warp_sum_arr);
if (threadIdx.x == 0)
    atomicAdd(trap_p, blk_result);
}
int main(void) {
    int n = 1 << 20;
    int th_per_blk = 1024;
    int blk_ct = ceil(n * 1.0 / th_per_blk);
    float a = 0.0, b = 1.0, x, *trap_p, trap;
    cudaMallocManaged(&trap_p, sizeof(float));
    struct timespec start, finish;
    timespec_get(&start, TIME_UTC);
    *trap_p = 0.5 * (f(a) + f(b));
    float h = (b - a) / n;
```

```
Dev_trap <<<blk_ct, th_per_blk>>> (a, b, h, n,
trap_p);
cudaDeviceSynchronize();
*trap_p = h * (*trap_p);
timespec_get(&finish, TIME_UTC);
printf("GPU time = %f\n", finish.tv_sec -
start.tv_sec + (finish.tv_nsec - start.tv_nsec) / 1e9);
timespec_get(&start, TIME_UTC);
trap = 0.5 * (f(a) + f(b));
for (int i = 1; i <= n - 1; i++) {
    x = a + i * h;
    trap += f(x);
}
trap = trap * h;
timespec_get(&finish, TIME_UTC);
printf("CPU time = %f\n", finish.tv_sec -
start.tv_sec + (finish.tv_nsec - start.tv_nsec) / 1e9);
printf("The area as computed by cuda is: %f\n",
*trap_p);
printf("The area as computed by cpu is: %f\n",
trap);
cudaFree(trap_p);
return 0;
}
```

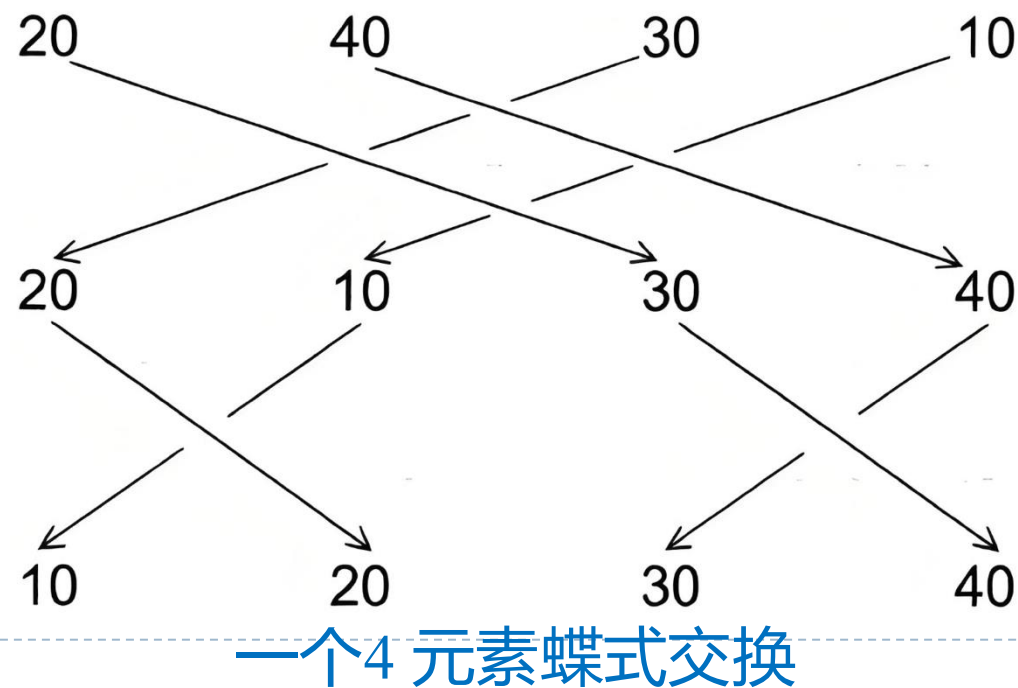
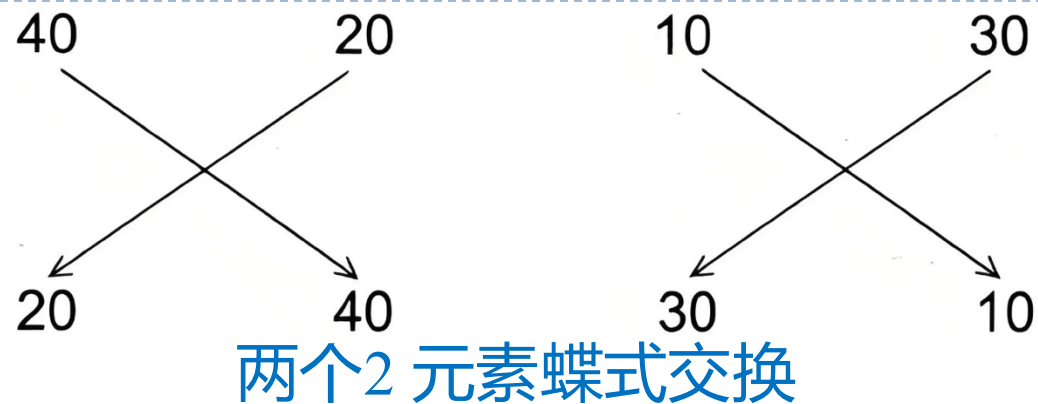

性能

■ 使用随机块大小的梯形法则的运行时间。

系统	ARM Cortex-A15	NVIDIA GK20A	Intel Core i7	NVIDIA GeForce GTX Titan X
时钟 SM 的数量, SP 的数量	2.3 GHz	852 MHz 1,192	3.5 GHz	1.08 GHz 24,3072
原始的	33.6	20.7	4.48	3.08
线程束洗牌, 32 ths/blk		14.4		0.210
共享内存, 32 ths/blk		15.0		0.206
线程束洗牌		12.8		0.141
共享内存		14.3		0.150

双调排序

- 串行双调排序
 - 4元素列表排序



4元素序列的双调排序

- 首先是两个2 元素蝶式交换，然后是一个4个元素蝶式交换。

	下标			
	0	1	2	3
开始时的列表	40	20	10	30
期望的两个元素顺序	递增		递减	
2 元素蝶式配对	1	0	3	2
比较交换后的列表	20	40	30	10
期望的四个元素顺序	递增			
4 元素蝶式：阶段 A 配对	2	3	0	1
阶段 A：比较交换后的列表	20	10	30	40
阶段 B 配对	1	0	3	2
阶段 B：比较交换后的列表	10	20	30	40

串行双调排序

■ 8元素列表排序

▶ 四个2元素蝶式交换

- 前4个元素构建一个双调序列，后4个元素构建另一个双调序列。

▶ 两个4元素蝶式交换

- 构建一个8元素双调序列，前4个元素递增，后4个元素递减。

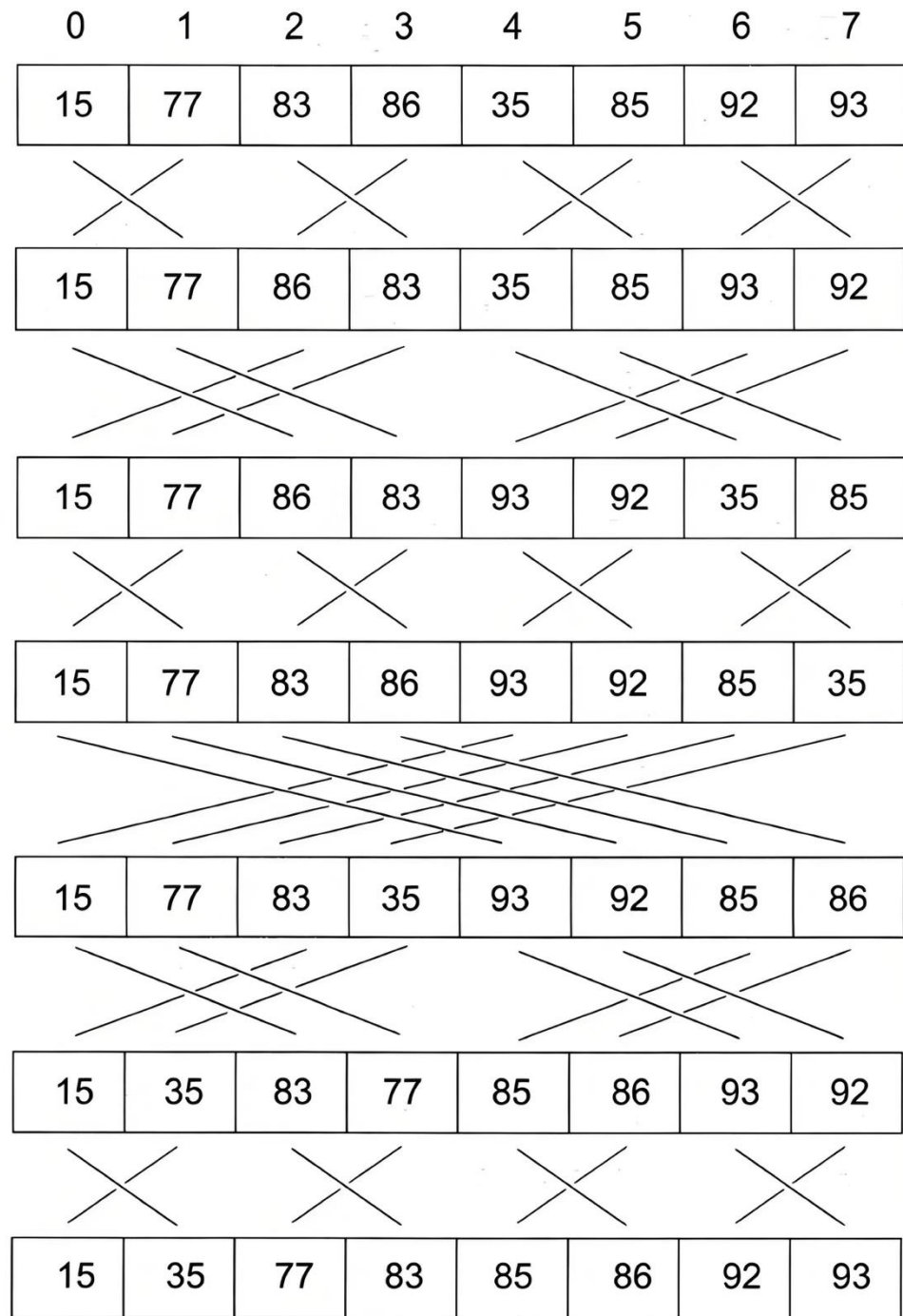
▶ 一个8元素蝶式交换

- 将8元素双调序列变成一个递增序列。

4元素
蝶式交换

2元素
蝶式交换

8元素
蝶式交换



	下标							
	0	1	2	3	4	5	6	7
开始时的列表	15	77	83	86	35	85	92	93
期望的两个元素顺序	递增		递减		递增		递减	
2 元素配对	1	0	3	2	5	4	7	6
比较交换后的列表	15	77	86	83	35	85	93	92
期望的四个元素顺序	递增				递减			
4 元素：阶段 A 配对	2	3	0	1	6	7	4	5
阶段 A：比较交换后的列表	15	77	86	83	93	92	35	85
4 元素：阶段 B 配对	0	1	3	2	5	4	7	6
阶段 B：比较交换后的列表	15	77	83	86	93	92	85	35
期望的八个元素顺序	递增							
8 元素：阶段 A 配对	4	5	6	7	0	1	2	3
阶段 A：比较交换后的列表	15	77	83	35	93	92	85	86
8 元素：阶段 B 配对	2	3	0	1	6	7	4	5
阶段 B：比较交换后的列表	15	35	83	77	85	86	93	92
8 元素：阶段 C 配对	1	0	3	2	5	4	7	6
阶段 C：比较交换后的列表	15	35	77	83	85	86	92	93

串行双调排序

- 使用蝶式结构为比较交换选择“配对”。
- 在递增和递减序列之间交替。
- 大小为 m 的蝶式交换需要 $\log_2 m$ 个阶段来排序。
- Get_pair函数确定“线程”负责哪一对元素。
- Increasing_seq函数确定列表中的一对元素应该按升序还是降序排列

```
。    for (bf_sz = 2; bf_sz <= n; bf_sz = 2*bf_sz)
        for (stage = bf_sz/2; stage > 0; stage = stage/2)
            for (th = 0; th < n/2; th++) {
                Get_pair(th, stage, &my_elt1, &my_elt2);
                if (Increasing_seq(my_elt1, bf_sz))
                    Compare_swap(list, my_elt1, my_elt2, INC);
                else
                    Compare_swap(list, my_elt1, my_elt2, DEC);
            }
```

串行双调排序

```
#include <iostream>
#include <cmath>
using namespace std;
int main() {
    const int n = 32;
    int a[n];
    srand(time(NULL));
    for (int i = 0; i < n; i++)
        a[i] = rand() % 100;
    for (int i = 0; i < n; i++)
        cout << a[i] << " ";
    cout << endl;
    for (int bf_sz = 2; bf_sz <= n; bf_sz = 2 *
        bf_sz)
```

```
    for (int stage = bf_sz / 2; stage > 0; stage
        = stage / 2)
        for (int th = 0; th < n / 2; th++) {
            int i = (th / stage) * 2 * stage + th % stage;
            int j = i + stage;
            if (i / bf_sz % 2 == 0 && a[i] > a[j] || i /
                bf_sz % 2 == 1 && a[i] < a[j])
                swap(a[i], a[j]);
        }
    for (int i = 0; i < n; i++)
        cout << a[i] << " ";
    cout << endl;
}
```


蝶式交换和二进制表示

■ stage=1

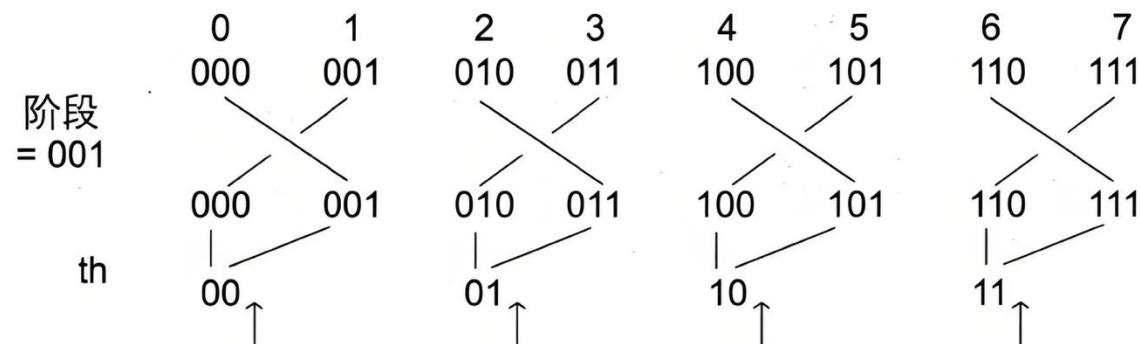
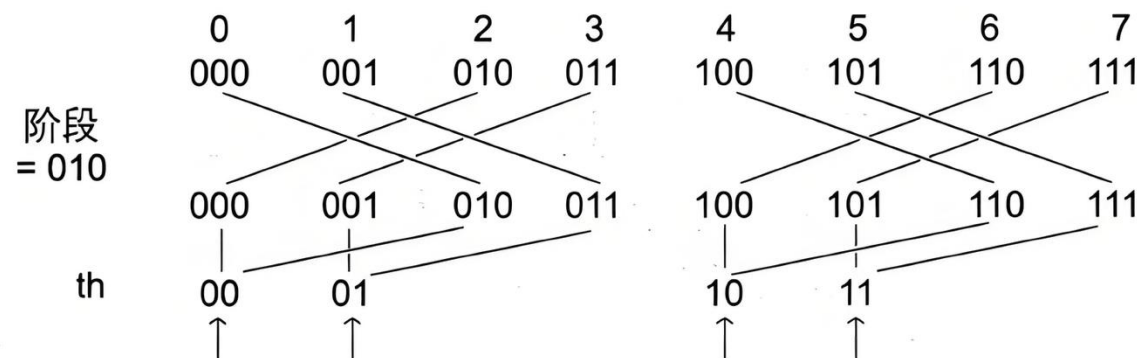
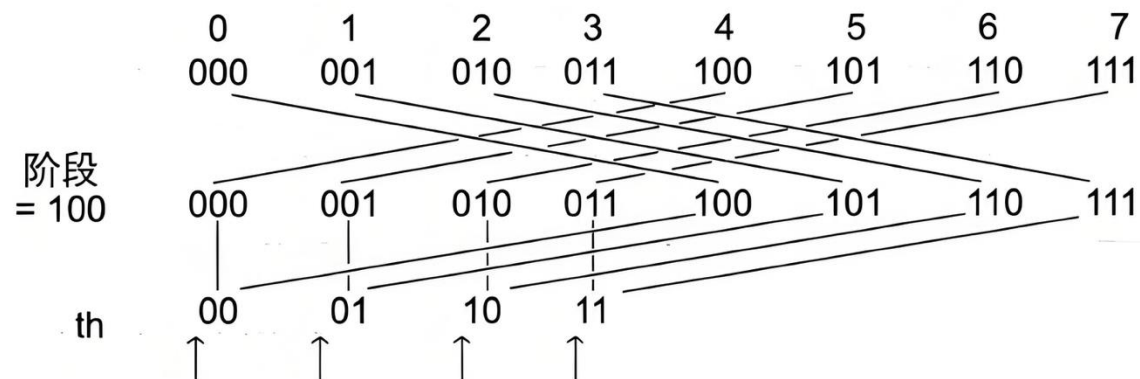
- ▶ 在线程号二进制位0的右边添加一位来获得元素。

■ stage=2

- ▶ 在线程号二进制位0和位1之间添加一位来获得元素。

■ stage=4

- ▶ 在线程号二进制位1和位2之间添加一位来获得元素。



双调排序中用到的不同值的二进制表示

蝶式交换和二进制表示

- 引入一个表示 $\log(\text{stage})$ 的变量`which_bit`
 - 附加位会立即插入位`which_bit`的右侧。
- `Insert_zero`
 - 将向线程序列号的二进制表示中添加一个0位。

双调排序中线程和元素的映射

阶段	线程	第一个元素	第二个元素
100 = 4	00 = 0	000 = 0	100 = 4
	01 = 1	001 = 1	101 = 5
	10 = 2	010 = 2	110 = 6
	11 = 3	011 = 3	111 = 7
010 = 2	00 = 0	000 = 0	010 = 2
	01 = 1	001 = 1	011 = 3
	10 = 2	100 = 4	110 = 6
	11 = 3	101 = 5	111 = 7
001 = 1	00 = 0	000 = 0	001 = 1
	01 = 1	010 = 2	011 = 3
	10 = 2	100 = 4	101 = 5
	11 = 3	110 = 6	111 = 7

蝶式交换和二进制表示

- 一对元素是否应该交换仅取决于元素位置和蝶式交换的大小。
- 元素下标和bf_sz的按位与。
 - ▶ 当按位与为0时，递增序列；否则递减序列。

元素	bf_sz		
	0010 = 2	0100 = 4	1000 = 8
0000 = 0	Inc	Inc	Inc
0001 = 1	Inc	Inc	Inc
0010 = 2	Dec	Inc	Inc
0011 = 3	Dec	Inc	Inc
0100 = 4	Inc	Dec	Inc
0101 = 5	Inc	Dec	Inc
0110 = 6	Dec	Dec	Inc
0111 = 7	Dec	Dec	Inc

第二个双调排序的串行实现的伪代码

```
for (bf_sz = 2; bf_sz <= n; bf_sz = 2*bf_sz)
    for (stage = bf_sz/2; stage > 0; stage = stage/2) {
        which_bit = Which_bit(stage);
        for (th = 0; th < n/2; th++) {
            my_elt1 = Insert_zero(th, which_bit);
            my_elt2 = my_elt1 ^ stage;
            Compare_swap(list, my_elt1, my_elt2, my_elt1 & bf_sz);
        }
    }
```

并行双调排序I

- 假设目前线程0和1处于不同的线程束中，因此它们不会自动同步。
- 线程0的第二个Compare_swap应该发生在线程1的第一个Compare_swap之后，并且因为它们的执行顺序被颠倒了，所以最终的列表不是有序的。

时间	线程 0		线程 1		阶段后的列表
	阶段	执行	阶段	执行	
0	—	—	—	—	20, 40, 30, 10
1	2	Compare_swap(elt 0, elt 2)	—	Idle	20, 40, 30, 10
2	1	Compare_swap(elt 0, elt 1)	—	Idle	20, 40, 30, 10
3	—	空闲	2	Compare_swap(elt 1, elt 3)	20, 10, 30, 40
4	—	空闲	1	Compare_swap(elt 2, elt 3)	20, 10, 30, 40

并行双调排序I

- 如果有多个线程束，就会产生这个问题。
- 单个线程块中的线程可以通过调用__syncthreads来完成此操作。

```
unsigned th = threadIdx.x;
for (bf_sz = 2; bf_sz <= n; bf_sz = 2*bf_sz) {
    for (stage = bf_sz/2; stage > 0; stage = stage/2) {
        which_bit = Which_bit(stage);
        my_elt1 = Insert_zero(th, which_bit);
        my_elt2 = my_elt1 ^ stage;
        Compare_swap(list, my_elt1, my_elt2, my_elt1 & bf_sz);
        __syncthreads();
    }
}
```

并行双调排序I

```
#include <algorithm>
#include <stdio.h>
#include <cuda.h>
__device__ unsigned Insert_zero(unsigned val, unsigned j) {
    unsigned left_bits, right_bits, left_ones,
    right_ones;
    left_ones = (~0) << j;
    right_ones = ~left_ones;
    left_bits = left_ones & val;
    right_bits = right_ones & val;
    return (left_bits << 1) | right_bits;
}
__device__ void Compare_swap(float a[], unsigned elt, unsigned partner, unsigned inc_dec) {
    float tmp;
    if (inc_dec == 0 && a[elt] > a[partner] ||
    inc_dec != 0 && a[elt] < a[partner]) {
        tmp = a[elt];
        a[elt] = a[partner];
        a[partner] = tmp;
    }
}
```

```
}
__global__ void Pbitonic_sort(float a[], int n) {
    unsigned bf_sz, stage, my_elt1, my_elt2,
    initial_bit, which_bit;
    unsigned th = threadIdx.x;
    for (bf_sz = 2, initial_bit = 0; bf_sz <= n;
    bf_sz = bf_sz << 1, initial_bit++)
    for (stage = bf_sz >> 1, which_bit =
    initial_bit; stage > 0; stage = stage >> 1,
    which_bit--) {
        my_elt1 = Insert_zero(th, which_bit);
        my_elt2 = my_elt1 ^ stage;
        Compare_swap(a, my_elt1, my_elt2,
        my_elt1 & bf_sz);
        __syncthreads();
    }
}
int main(void) {
    int n = 2048;
    float* a, * b;
    cudaMallocManaged(&a, n * sizeof(float));
    b = (float*)malloc(n * sizeof(float));
}
```

```
srand(time(NULL));
for (int i = 0; i < n; i++)
    a[i] = b[i] = rand() * 1.0 / RAND_MAX;
struct timespec start, finish;
timespec_get(&start, TIME_UTC);
Pbitonic_sort <<<1, n / 2>>> (a, n);
cudaDeviceSynchronize();
timespec_get(&finish, TIME_UTC);
printf("GPU time = %f\n", finish.tv_sec -
start.tv_sec + (finish.tv_nsec - start.tv_nsec)
/ 1e9);
timespec_get(&start, TIME_UTC);
std::sort(b, b + n);
timespec_get(&finish, TIME_UTC);
printf("CPU time = %f\n", finish.tv_sec -
start.tv_sec + (finish.tv_nsec - start.tv_nsec)
/ 1e9);
for (int i = 0; i < n; i++)
    if (a[i] != b[i])
        printf("error\n");
cudaFree(a);
free(b);
}
```

并行双调排序I

- 排序列表的大小为 $2048m$ 。
- 给每个线程分配 $2m$ 个元素。
- 合并两个长度为 m 的子列表。
- 处理递增的子列表。
 - 将合并后的长度为 $2m$ 的子列表分为两个长度为 m 的子列表。
- 处理递减的子列表
 - 相反地分配。

并行双调排序I

```
#include <algorithm>
#include <stdio.h>
#include <cuda.h>

__device__ unsigned Insert_zero(unsigned val,
unsigned j) {
    unsigned left_bits, right_bits, left_ones,
    right_ones;
    left_ones = (~0) << j;
    right_ones = ~left_ones;
    left_bits = left_ones & val;
    right_bits = right_ones & val;
    return (left_bits << 1) | right_bits;
}

__device__ void Compare_swap(float a[],
unsigned elt, unsigned partner, unsigned
inc_dec, unsigned m) {
    unsigned u, v;
    for (unsigned i = 0; i < 2 * m - 1; i++)
    for (unsigned j = 0; j < 2 * m - i - 1; j++) {
        u = j < m ? elt * m + j : partner * m + j - m;
        v = j + 1 < m ? elt * m + j + 1 : partner * m + j
        + 1 - m;
        float tmp;
        if (inc_dec == 0 && a[u] > a[v] || inc_dec != 0
        && a[u] < a[v]) {
            tmp = a[u];
```

```
            a[u] = a[v];
            a[v] = tmp;
        }
    }
}

__global__ void Pbitonic_sort(float a[], int n,
unsigned m) {
    unsigned bf_sz, stage, my_elt1, my_elt2,
    initial_bit, which_bit;
    unsigned th = threadIdx.x;
    for (bf_sz = 2, initial_bit = 0; bf_sz <= n / m;
    bf_sz = bf_sz << 1, initial_bit++)
    for (stage = bf_sz >> 1, which_bit = initial_bit;
    stage > 0; stage = stage >> 1, which_bit--) {
        my_elt1 = Insert_zero(th, which_bit);
        my_elt2 = my_elt1 ^ stage;
        Compare_swap(a, my_elt1, my_elt2, my_elt1
        & bf_sz, m);
        __syncthreads();
    }
}

int main(void) {
    unsigned m = 3;
    int n = 2048 * m;
    float* a, * b;
    cudaMallocManaged(&a, n * sizeof(float));
```

```
    b = (float*)malloc(n * sizeof(float));
    srand(time(NULL));
    for (int i = 0; i < n; i++)
        a[i] = b[i] = rand() * 1.0 / RAND_MAX;
    struct timespec start, finish;
    timespec_get(&start, TIME_UTC);
    Pbitonic_sort <<<1, n / m / 2>>> (a, n, m);
    cudaDeviceSynchronize();
    timespec_get(&finish, TIME_UTC);
    printf("GPU time = %f\n", finish.tv_sec -
    start.tv_sec + (finish.tv_nsec - start.tv_nsec) /
    1e9);
    timespec_get(&start, TIME_UTC);
    std::sort(b, b + n);
    timespec_get(&finish, TIME_UTC);
    printf("CPU time = %f\n", finish.tv_sec -
    start.tv_sec + (finish.tv_nsec - start.tv_nsec) /
    1e9);
    for (int i = 0; i < n; i++)
        if (a[i] != b[i])
            printf("error\n");
    cudaFree(a);
    free(b);
}
```

并行双调排序II

- $n = 2 \times \text{blk_ct} \times \text{th_per_blk}$
- 首先使用双调排序将每个长度为 $2 \times \text{th_per_blk}$ 的子列表排序为递增或递减列表。
- 让几组线程块合作形成排序子列表，长度依次为 $4 \times \text{th_per_blk}$, $8 \times \text{th_per_blk}$, ...
 - ▶ 在单个核函数中执行当前蝶式交换的一个阶段。
 - ▶ 通过从核函数返回然后再次调用核函数进行下一阶段来同步不同块中的线程。
 - ▶ 在核函数调用之间不需要调用 `cudaDeviceSynchronize`。
- 在重复执行“单阶段”核函数之后，最终将“子蝶式交换”应用于长度 $\leq 2 \times \text{th_per_blk}$ 的子列表，线程块可以彼此独立工作。

实现通用CUDA双调排序的主机函数

```
/* 对包含2* th_per_blk元素的子列表进行排序 */
Pbitonic_start <<<blk_ct, th_per_blk>>> (list, n);

for (bf_sz = 4*th_per_blk; bf_sz <= n; bf_sz = bf_sz/2) {
    for (stage = bf_sz/2; stage >= 2*th_per_blk;
         stage = stage/2) {
        /* 完成蝶式的单个阶段 */
        Pbutterfly_one_stage <<<blk_ct, th_per_blk>>> (list, n,
                                                         bf_sz, stage);
    }
    /* 通过使用“短”子列表来完成当前的蝶式 */

    Pbutterfly_finish <<<blk_ct, th_per_blk>>> (list, n,
                                                  bf_sz);
}
```

并行双调排序II

- 8个线程块，其中每个线程块由1024个线程组成，对有 $2 \times 8 \times 1024 = 16384$ 个元素的列表进行排序。
 - ▶ 首先，每个线程块将大小为2, 4, ..., 2048的蝶式交换进行迭代。
 - ▶ 然后，续执行主机的for 循环，对bf_sz值为4096, 8192, 16384 时进行迭代。
 - 在执行完内层循环时，没有完成一次“蝶式交换”，并且还需要完成阶段1024, 512, ..., 4, 2, 1 的蝶式交换。

并行双调排序II

调用 Pbitonic_start 后已排好序的子列表中的元素

块	被排序元素			
0	0	1	...	2 047
1	2 048	2 049	...	4 095
⋮	⋮	⋮		⋮
8	14 436	14 437	...	16 383

由 Pbutterfly_one_stage 执行的阶段

bf_sz	阶段		
4 096	2 048		
8 192	4 096	2 048	
16 384	8 192	4 096	2 048

CUDA双调排序的性能

含有 2 048 个整数的列表的不同排序平均运行时间 (ms)

系统	Intel Xeon 4116	Intel Xeon 4116	Nvidia Quadro P5000
时钟	2.1GHz	2.1GHz	1.73 GHz
SM 的数量, SP 的数量			20, 1 280
排序种类	qsort	双调排序	并行双调排序
块数, 线程数			1, 1 024
运行时间	0.116	1.19	0.128

含有 2 097 152 个整数的列表上不同排序的平均运行时间

系统	Intel Xeon 4116	Intel Xeon 4116	NVIDIA Quadro P5000
时钟	2.1GHz	2.1GHz	1.73GHz
SM 的数量, SP 的数量			20, 1 280
排序种类	qsort	双调排序	并行双调排序
块数, 线程数			1 024, 1 024
运行时间	539	3 710	13.1

小结

- 图形处理单元通用计算 (GPGPU)
- 单指令流多数据流 (SIMD)
- 流处理器 (SP)
- 流式多处理器 (SM)
- 线程
- 线程块
- 线程束
- 共享内存
- 全局内存

小结

- 核函数
- 设备
- 异构编程
- 网格
- 异步
- 统一内存
- 线程分歧

小结

- 栅栏
- 原子操作
- 线程束洗牌
- 通道
- 传播总和
- 共享内存被划分为库